

A Project Proposal Report on

Gudau: A Ride Sharing and On-Demand Services Application

A Project Report Submitted for the Course

Object-Oriented Programming (OOP)

Bachelor of Engineering in Information Technology

Submitted by:

Saugat Maharjan, WU248232

Date:

10/08/2026



King's College Nepal

Bijulibajar, Kathmandu

ABSTRACT

Gudau is a console-based C++ application that simulates a ride-sharing and on-demand services platform, in the spirit of apps such as Pathao or InDrive. The primary objective of the project is to provide a single, menu-driven interface through which a user can book a ride, order a food delivery, request a parcel delivery, or request a repair service, each with its own set of questions and fare calculation logic. A user enters their name and phone number once at start-up and can then repeatedly use any of the four services until they choose to exit. The system is designed around object-oriented principles: an abstract `Service` base class defines a common contract (`requestService()` and `displayDetails()`), and four concrete classes – `RideBooking`, `FoodDelivery`, `ParcelDelivery`, and `RepairService` – inherit from it and override this behaviour with their own questions and fare formulas. Each service is associated with a `User` object through composition rather than inheritance, since a service is requested by a user rather than being a kind of user. Fares are computed using distance- and weight-based formulas, and a confirmation is displayed to the user through polymorphic dispatch on a single `Service*` base pointer. The application is being developed as a single, self-contained `Gudau.cpp` file to keep the scope manageable for the semester while still demonstrating encapsulation, inheritance, polymorphism, abstraction, and composition in a working, testable program. This proposal outlines the problem being addressed, the objectives and scope of the project, a review of comparable ride-sharing platforms, the development methodology and tools to be used, the system design (class, use case, and sequence diagrams), the proposed deliverables, and a task/time schedule for completing the project.

Keywords: *Ride Sharing, On-Demand Services, Object-Oriented Programming, C++, Console Application*

TABLE OF CONTENTS

Abstract	i
Table of Contents	ii
List of Figures	iii
List of Tables	iv
1 INTRODUCTION	1
1.1 PROBLEM STATEMENT	1
1.2 PROJECT OBJECTIVES	1
1.3 SCOPE AND LIMITATIONS	2
2 LITERATURE REVIEW	3
3 METHODOLOGY	4
3.1 SOFTWARE DEVELOPMENT LIFECYCLE	4
3.1.1 Requirement Analysis	4
3.1.2 Design	4
3.1.3 Coding	4
3.1.4 Testing and Evaluation	5
3.2 TOOLS TO BE USED	5
3.3 OOP CONCEPTS TO BE APPLIED	5
4 SYSTEM DESIGN	7
4.1 CLASS DIAGRAM	7
4.2 USE CASE DIAGRAM	8
4.3 SEQUENCE DIAGRAM	9
5 PROPOSED DELIVERABLES	10
6 SAMPLE OUTPUT	11
7 PROJECT TASK AND TIME SCHEDULE	13
Bibliography	14

LIST OF FIGURES

1	Class Diagram of the Gudau System	7
2	Use Case Diagram of the Gudau System	8
3	Sequence Diagram of a Typical Gudau Session	9

LIST OF TABLES

1	Service Questions and Fare Formulas	10
2	Time Schedule	13

1. INTRODUCTION

This project carries the motive of providing a simplified, menu-driven way for a user to request everyday on-demand services – a ride, a food delivery, a parcel delivery, or a repair – from a single application, while giving the developer a practical setting in which to apply object-oriented programming concepts. In this section, the problem, objectives, scope, and limitations of the project are described in detail.

1.1 PROBLEM STATEMENT

On-demand services such as ride booking, food delivery, parcel delivery, and repair requests are usually handled by separate, unrelated applications or by direct phone calls, each with its own way of asking for information and calculating cost. This fragmentation makes it harder to model and reason about the underlying system, and it does not lend itself well to reuse: similar logic (asking for a destination, computing a distance-based fare, printing a confirmation) tends to be duplicated across each service rather than being shared.

From a learning perspective, this also makes such systems a poor example of object-oriented design if implemented naively, since the common structure that all four services share – a requester, a location, and a fare – is not factored out into a reusable base class.

1.2 PROJECT OBJECTIVES

To address the problems stated above, the Gudau console application is proposed with the following objectives:

1. **Provide a single interface for four on-demand services:** Allow a user to book a ride, order food delivery, request parcel delivery, or request a repair service, all from one main menu.
2. **Model the shared structure of a “service” using inheritance and polymorphism:** Define an abstract Service base class with virtual functions that each concrete service overrides, so that `main()` can operate on any service through a single `Service*` pointer.
3. **Compute accurate, service-specific fares:** Apply distance-based and weight-based fare formulas for rides, food deliveries, and parcel deliveries, and cor-

rectly handle special cases such as heavy parcels requiring a 4-wheeler.

4. **Demonstrate core OOP principles in a small, testable codebase:** Apply encapsulation, inheritance, polymorphism, abstraction, and composition in a single, compiled-and-tested `Gudau.cpp` file.

1.3 SCOPE AND LIMITATIONS

In the Gudau project, a console application will be developed such that a single user can register their details once and then repeatedly request any of the four supported services in a session.

The scope of this project is:

1. The target audience is a single console user per run, who can request a ride, food delivery, parcel delivery, or repair service.
2. The application computes and displays a fare (where applicable) based on distance, weight, and vehicle/transport type.
3. The application is implemented entirely in standard C++ as a single source file, with no external database or network dependency.

The limitations of this project are:

1. The application is console-based only; it does not provide a graphical or web interface.
2. There is no persistent storage – user and request data exist only for the duration of a single run.
3. Only one user account is handled per session; there is no multi-user or concurrent booking support.

2. LITERATURE REVIEW

In this section, existing ride-sharing and on-demand service platforms that provide similar functionality are briefly reviewed.

Commercial ride-sharing and on-demand platforms such as Pathao and InDrive provide mobile applications through which a user can book a ride, and in some cases order food or send a parcel, from a single account. These platforms typically rely on GPS-based location tracking, real-time driver matching, and dynamic (surge-based) pricing, and are backed by large server-side systems and databases. While such platforms are a useful reference for the kind of user flow Gudau aims to emulate (a single menu of on-demand services with fare calculation), their scale and reliance on live location data, payment gateways, and networked backend infrastructure place them well outside the scope of a single-semester, single-file C++ console project.

The key gap this project addresses is not competing with such platforms in functionality, but instead demonstrating – in a simplified, self-contained form – how the shared structure behind “a user requests a service and is charged a fare” can be modelled cleanly using object-oriented design, so that adding a new kind of service in the future would mean writing one new derived class rather than duplicating menu and fare logic.

3. METHODOLOGY

The following methodology will be adopted for a smooth development process, to meet the requirements of the project and not deviate from them.

3.1 SOFTWARE DEVELOPMENT LIFECYCLE

The project will follow the incremental model of software development. In this model, the development process is broken down into smaller increments, each involving Requirement Analysis, Design, Coding, and Testing, with each increment delivering a partial but usable version of the project.

In the first increment, the core product will be developed: user detail collection (name and phone number, asked once), the main menu loop, and one working service (Ride Booking) with its own question flow, fare calculation, and confirmation display.

In subsequent increments, the remaining services – Food Delivery, Parcel Delivery, and Repair Service – will be added as additional subclasses of `Service`, reusing the same menu loop and polymorphic dispatch established in the first increment.

3.1.1 Requirement Analysis

This is the first step, where the required services, the questions each one asks, and the fare formula for each are collected and analysed from the project's handwritten design. The outcome of this phase is a clear specification of each service's inputs and fare formula.

3.1.2 Design

In this phase, the system is designed based on the requirement analysis. Class, Use Case, and Sequence diagrams are developed to model the entities, their interactions, and the runtime flow of a typical user session.

3.1.3 Coding

The functionalities specified in the requirement analysis and modelled during the design phase are implemented in C++, as a single `Gudau.cpp` file, with each service

represented by its own derived class.

3.1.4 Testing and Evaluation

In this phase, each service is tested individually by running the application and stepping through its question flow and fare calculation. The entire system is also tested end-to-end, from user registration through the main menu to exit, to confirm that the menu loop, polymorphic dispatch, and object cleanup all work correctly across repeated uses.

3.2 TOOLS TO BE USED

The following tools will be used throughout this project to streamline the development process.

VS CODE:

Visual Studio Code will be used as the primary code editor for writing and debugging the C++ source code.

GIT AND GITHUB:

Git is a distributed version control system that will be used to manage different versions of the project. GitHub will be used to host the project repository and track changes over time.

C++ COMPILER (g++):

A standard C++ compiler will be used to compile and run `Gudau.cpp` locally throughout development and testing.

3.3 OOP CONCEPTS TO BE APPLIED

The following object-oriented programming concepts will be applied during the design and implementation of Gudau:

Encapsulation:

The internal data of the `User` and `Service` classes (such as name, phone, location, and fare) is kept private or protected and is only reachable through constructors and public getter functions. No data is left directly editable from outside its class.

Inheritance:

Service is the abstract base class. RideBooking, FoodDelivery, ParcelDelivery, and RepairService all derive from it. This hierarchy is used because every one of Gudau's offerings is a kind of service that a user requests, that has a location and possibly a fare, and that closes out – a shared shape that belongs in one base class instead of being duplicated four times.

Polymorphism:

Service declares `virtual void requestService()` and `virtual void displayDetails() const`. Each derived class overrides these with its own logic. `main()` calls both through a single `Service*` base pointer, so the same line of code correctly runs different behaviour depending on which object it actually points to at runtime.

Abstraction:

Service has pure virtual functions, making it an abstract class. It cannot be instantiated on its own – it only exists to define a common contract that the concrete services must fulfil.

Constructors and Destructors:

Every class initializes its members in its constructor, avoiding uninitialized data. Each derived service also defines its own destructor that prints a closing message, showing that derived destructors run before the base class destructor.

Composition:

Service holds a `User*` – a service has a user who requested it, rather than being a user. This is composition/association, not inheritance, because a RideBooking is not a kind of User.

4. SYSTEM DESIGN

This section provides a comprehensive overview of the proposed system's architecture, focusing on three key diagrams: the Class diagram, the Use Case diagram, and the Sequence diagram. These diagrams illustrate the core components of the system and the interactions between them.

4.1 CLASS DIAGRAM

The class diagram below shows the abstract `Service` base class, its four derived classes (`RideBooking`, `FoodDelivery`, `ParcelDelivery`, `RepairService`), and its association with the `User` class.

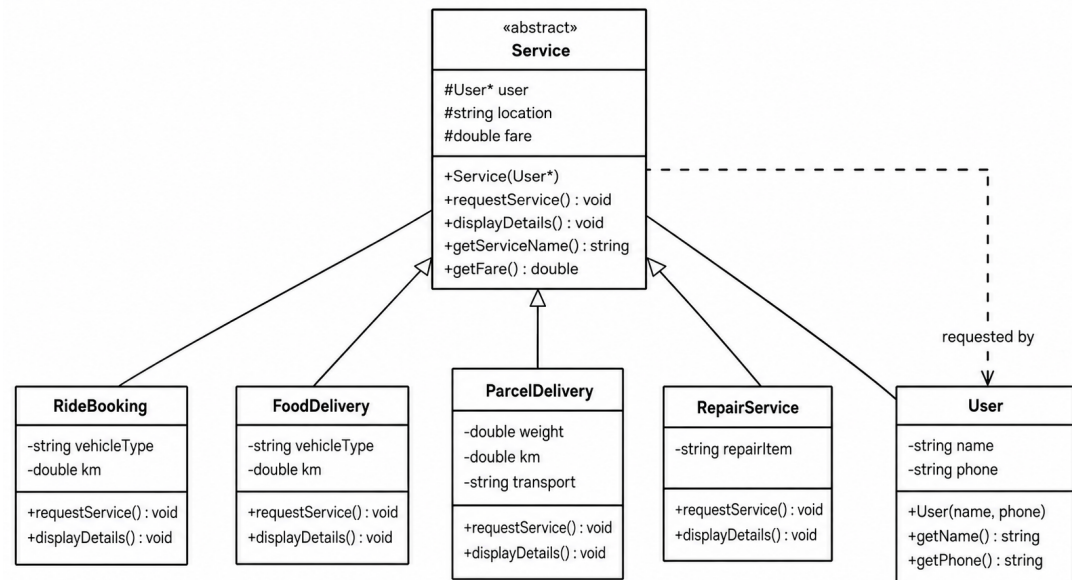


Figure 1: Class Diagram of the Gudau System

4.2 USE CASE DIAGRAM

The use case diagram below shows the interactions between the User actor and the Gudau system: registering, requesting one of the four services, viewing the confirmation and fare, and exiting.

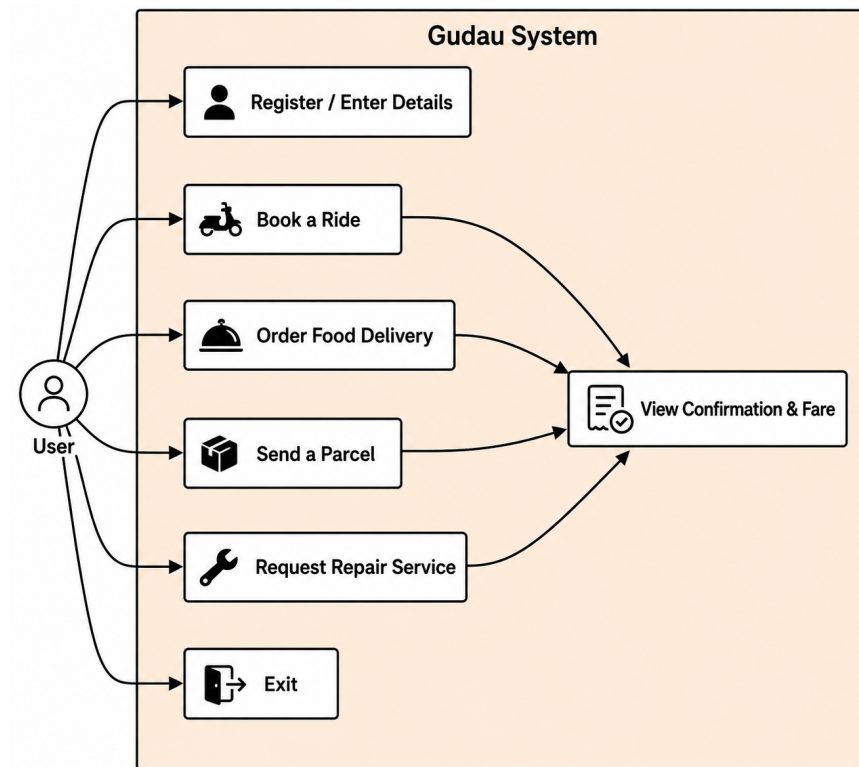


Figure 2: Use Case Diagram of the Gudau System

4.3 SEQUENCE DIAGRAM

The sequence diagram below illustrates a typical session: the user enters their details, selects a service from the main menu, answers the service-specific questions, and receives a confirmation with the computed fare, after which the service object is deleted and the main menu is shown again.

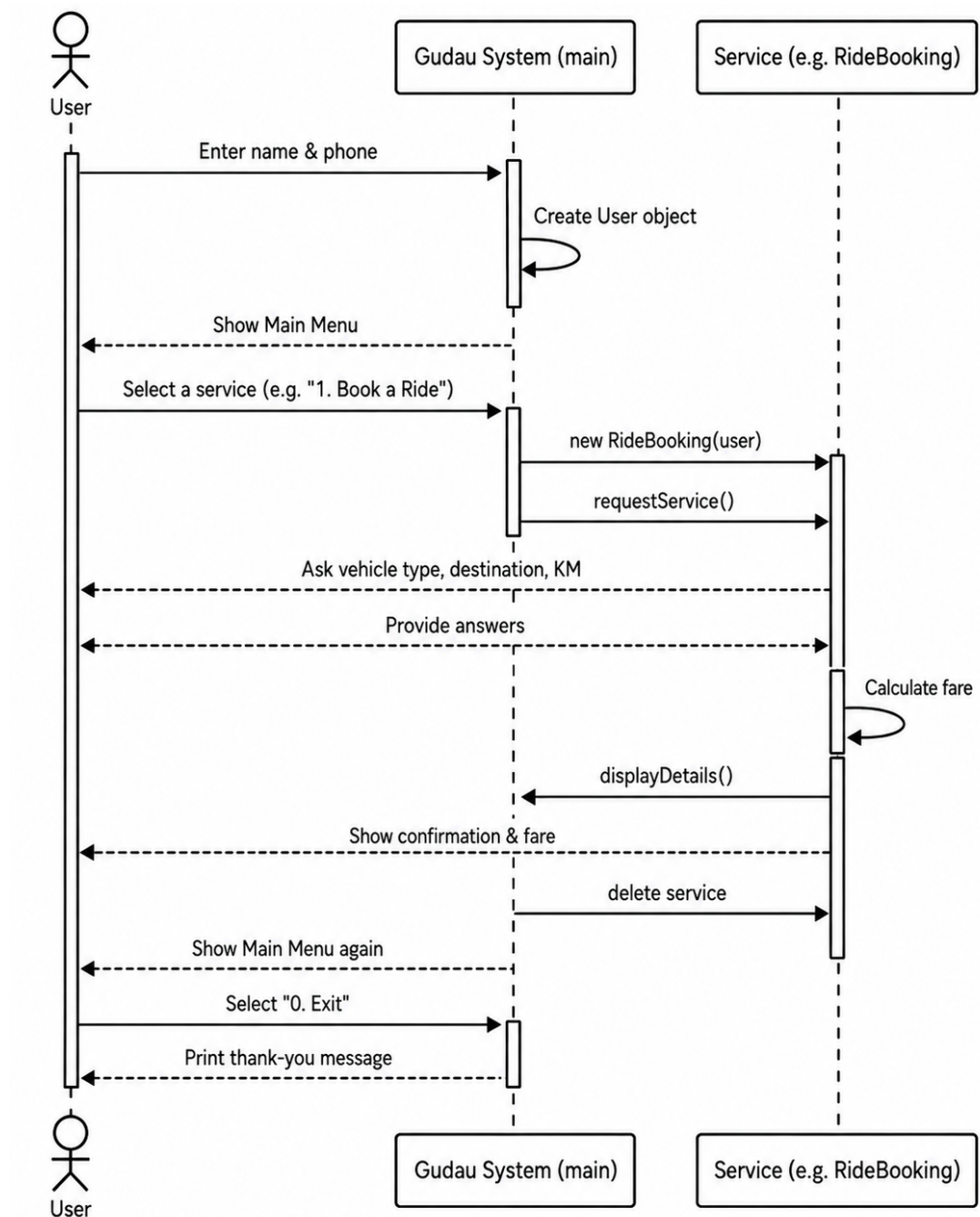


Figure 3: Sequence Diagram of a Typical Gudau Session

5. PROPOSED DELIVERABLES

The project will deliver a single, self-contained console application, `Gudau.cpp`, that a user runs from a terminal. There is essentially one user role in the application: the person using the console, who is asked for their name and phone number once and can then request any of the four supported services.

Book a Ride: The user selects Bike or Scooter, and enters a destination and distance in kilometres; the fare is calculated at NPR 16/km.

Food Delivery: The user selects Cycle or Scooter, and enters a destination and distance in kilometres; the fare is calculated at NPR 9/km.

Parcel Delivery: The user enters the parcel weight and destination. The fare is calculated per kilometre based on weight: NPR 12/km for parcels under 10 kg, NPR 14/km for parcels under 20 kg, and NPR 18/km for parcels of 20 kg or more, with the default Scooter automatically replaced by a 4-Wheeler and a “heavy load” message shown for the heaviest tier.

Repair Service: The user enters their location and freely describes what needs repairing (e.g. “Puncture”, “Engine noise”). No fare is charged; the system simply confirms that a repairman is on the way.

Table 1 summarizes the questions asked and fare formula for each service.

Table 1: Service Questions and Fare Formulas

Service	Questions Asked	Fare Formula
Book a Ride	Bike or Scooter, destination + KM	NPR 16/km
Food Delivery	Cycle or Scooter, destination + KM	NPR 9/km
Parcel Delivery	Weight (kg), destination + KM	<10kg: NPR 12/km, <20kg: NPR 14/km, ≥20kg: NPR 18/km (4-Wheeler + “heavy load” message)
Repair Service	Location, what needs repair	No fare – confirms a repairman is coming

6. SAMPLE OUTPUT

To validate the feasibility of the proposed design, an early prototype of `Gudau.cpp` has been compiled and run. Listing 1 shows a representative session covering user registration, a ride booking, and a parcel delivery.

Listing 1: Sample output captured from compiling and running the Gudau prototype

```
=====
                WELCOME TO GUDAU
            Ride Sharing & Services
=====

Enter your name: Sita Rai
Enter your phone number: 9812345678

Welcome, Sita Rai! Let's get you started.

===== GUDAU MAIN MENU =====
1. Book a Ride
2. Food Delivery
3. Parcel Delivery
4. Repair Service
0. Exit
=====
Enter your choice: 1

----- Book a Ride -----
    1. Bike
    2. Scooter
    Choose an option (1-2): 1
Where do you want to go? Thamel
Enter distance in KM: 5

===== Ride Booking Confirmed =====
Passenger      : Sita Rai (9812345678)
Vehicle        : Bike
Destination    : Thamel (5 KM)
Fare           : NPR 80.00
=====
[System] Closing Ride Booking request.

===== GUDAU MAIN MENU =====
...
Enter your choice: 3

----- Parcel Delivery -----
How much does the parcel weigh (in KG)? 25
```



```
Where should the parcel go? Bhaktapur
Enter distance in KM: 10
Your load is heavy so, 4 Wheeler is coming to pick.

==== Parcel Delivery Confirmed ====
Sender      : Sita Rai (9812345678)
Transport   : 4 Wheeler
Deliver to  : Bhaktapur (10.00 KM)
Parcel Weight : 25.00 KG
Fare        : NPR 180.00
=====
[System] Closing Parcel Delivery request.

===== GUDAU MAIN MENU =====
...
Enter your choice: 0

=====
      Thank you for using GUDAU!
      Have a safe journey!
=====
```

7. PROJECT TASK AND TIME SCHEDULE

The project time schedule is designed considering the requirements and constraints involved. Requirement analysis and design are given more emphasis early on to ensure the class hierarchy is correct before coding all four services.

Table 2: Time Schedule

TASK	APPROX DURATION IN DAYS
Requirements analysis (services, questions, fare formulas)	5
System Design (Class, Use Case, Sequence diagrams)	4
Coding – Core (Service, User, RideBooking, main menu loop)	6
Coding – Remaining services (Food, Parcel, Repair)	6
Testing and Debugging Individual Services	4
Overall System Test and Debugging	3
Documentation	5

BIBLIOGRAPHY

- Reger S. Pressman, Ph.D. Seventh Edition, *Software Engineering: A Practitioner's Approach*, McGraw Hill, 2010
- Stroustrup, Bjarne, *The C++ Programming Language*, 4th Edition, Addison-Wesley, 2013
- <https://cplusplus.com/doc/>
- <https://git-scm.com/doc>
- <https://code.visualstudio.com/docs>